

PHP – Using Databases

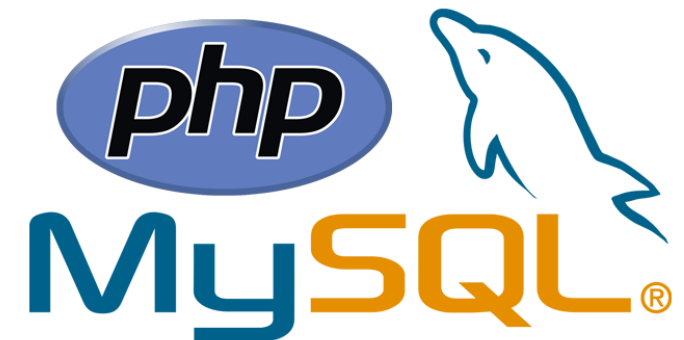
Sistemas de Informação 1

Credits and Disclaimer

- This material/slides are adapted from:
 - SINF1 2022/23 slides produced by Constantino Martins
 - ARQSI 2014/15 slides
 - Books
 - Web sites:
 - <https://www.php.net/>;
 - <https://www.w3schools.com/>
 - ...

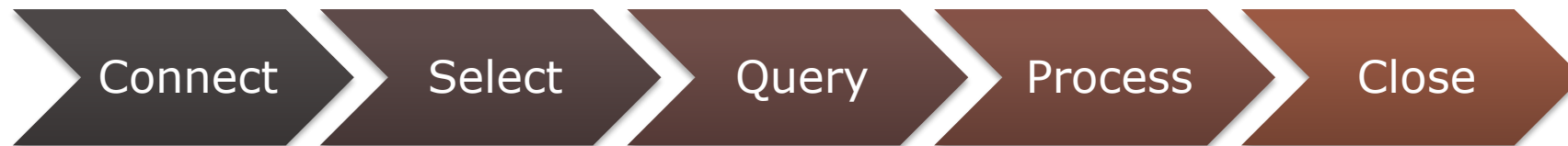
Using Databases

- Relational databases are the primary choice for data storage in the Web.
- A DBMS provides a database along with a set of tools to manage the data in the database.
 - One example for DBMS is MySQL.
- With PHP, you can connect to and manipulate databases. MySQL is the most popular database system used with PHP.



Using Databases

- To access a database, you need to:
 - Establish or open a connection to the DBMS server;
 - Select a database;
 - Execute the query against the database;
 - Process the result returned by the server;
 - Close the connection.



What is MySQL?

- MySQL is:
 - a database system used on the web
 - a database system that runs on a server
 - ideal for both small and large applications
 - very fast, reliable, and easy to use
 - based on standard SQL
 - is free to download and use (Url: <https://www.mysql.com/>)
 - is developed, distributed, and supported by Oracle Corporation
 - is named after co-founder Monty Widenius's daughter: *My*

MySQL PHP APIs

■ 3 APIs to connect to MySQL

- **PHP's MySQL extension** - PHP provides in-built support in the form of predefined functions to work with MySQL databases;
- **PHP's MySQLi extension** - MySQLi is an improved PHP extension, designed for interacting with MySQL databases, offering both procedural and object-oriented interfaces
- **PHP Data Objects (PDO)** – This extension defines a lightweight, consistent interface for accessing databases in PHP. PDO provides a data-access abstraction layer, which allows to use the same functions to issue queries and fetch data regardless of the used DBMS.

Why MySQLi?

- MySQLi introduces some of the following benefits:
 - Object-oriented interface;
 - Support for Prepared Statements;
 - Support for Multiple Statements;
 - Support for Transactions;
 - Enhanced debugging capabilities;
 - Embedded server support.

ISEP

MySQLi

MySQLi - Introduction

- The MySQLi extension, or as it is sometimes known, the MySQL improved extension, was developed to take advantage of new features found in MySQL systems versions 4.1.3 and newer.
- The MySQLi extension is included with PHP versions 5 and later.
- As well as the **object-oriented interface** the extension also provides a **procedural interface**.
- The MySQLi extension is built using the PHP extension framework, its source code is in the directory `ext/mysqli`.

MySQLi - Procedural Interface

- The procedural interface is like that of the old MySQL extension. In many cases, the function names differ only by prefix.
- Some MySQLi functions:

Function	Description
<code>mysqli_connect(host, username, password)</code>	Establishes a connection with the MySQL Server
<code>mysqli_select_db(link, dbname)</code>	Selects the specified database
<code>\$result = mysqli_query(link, query)</code>	Executes the specified query against the database and returns the result
<code>mysqli_fetch_assoc(\$result)</code>	Returns an associative array of the next row
<code>mysqli_num_rows(\$result)</code>	Returns the number of rows in the result
<code>mysqli_num_fields(\$result)</code>	Returns the number of columns in the result row

MySQLi - Procedural Interface

```
<?php
```

```
$mysqli = mysqli_connect("example.com", "user", "password", "database");
```

```
$result = mysqli_query($mysqli, "SELECT * FROM dataset");
```

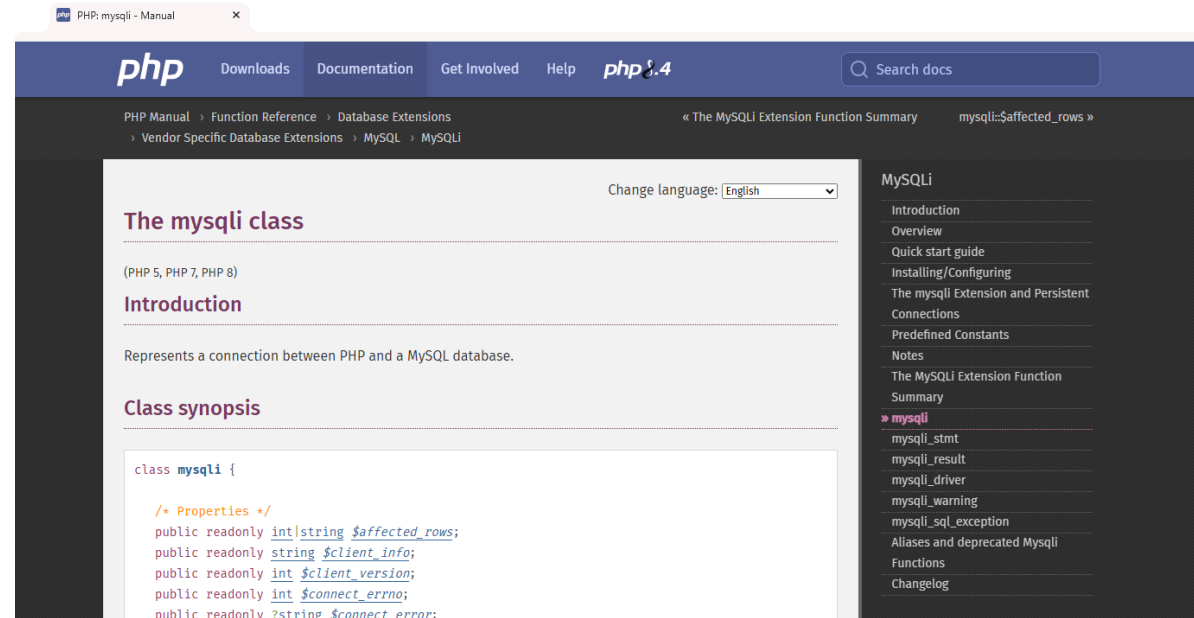
```
$row = mysqli_fetch_assoc($result);
```

```
echo $row['_msg'];
```

```
?>
```

MySQLi - Object-oriented Interface

- The object-oriented interface shows functions grouped by their purpose, making it easier to get started.
- The **mysqli** class represents a connection between PHP and a MySQL database.
- More details:



MySQLi - Object-oriented Interface

- Some `mysqli` methods and properties:

Method / Property	Description
<code>Mysqli::__construct()</code>	Establishes a connection with the MySQL Server
<code>mysqli::select_db(dbname)</code>	Selects the specified database
<code>mysqli::query(query)</code>	Executes the specified query against the database and returns the result
<code>mysqli_result::fetch_assoc()</code>	Returns an associative array of the next row
<code>\$mysqli_result::num_rows</code>	Returns the number of rows in the result
<code>\$mysqli_result::field_count</code>	Returns the number of columns in the result row

MySQLi - Object-oriented Interface

```
<?php
```

```
$mysqli = new mysqli("example.com", "user", "password", "database");
```

```
$result = $mysqli->query("SELECT * FROM dataset");
```

```
$row = $result->fetch_assoc($result);
```

```
echo $row['_msg'];
```

```
?>
```

ISEP

MySQLi Applications

Connecting a database

```
<?php
```

```
// Connect to the Database
```

```
$DB_NAME = 'DATABASE_NAME';
```

```
$DB_HOST = 'DATABASE_HOST';
```

```
$DB_USER = 'DATABASE_USER';
```

```
$DB_PASS = 'DATABASE_PASSWORD';
```

```
$mysqli = new mysqli($DB_HOST, $DB_USER, $DB_PASS, $DB_NAME);
```

```
if($mysqli->connect_errno) {
```

```
    echo "Connect failed: " . $mysqli->connect_error;
```

```
    (...)
```

```
}
```

```
?>
```


Querying a database

```
<?php
(...)
$query = 'SELECT * FROM datatable';

// $recordset is a set of records with the result of the query
$recordset = $mysqli->query($query);

if($recordset->num_rows > 0) {
    while( $row = $recordset->fetch_assoc() ) {
        echo $row['name'] . "-" . $row['age'];
    }
}

(...)
?>
```

Each row is an associative array indexed by the names of the DB fields, created by the method `fetch_assoc()`

ISEP

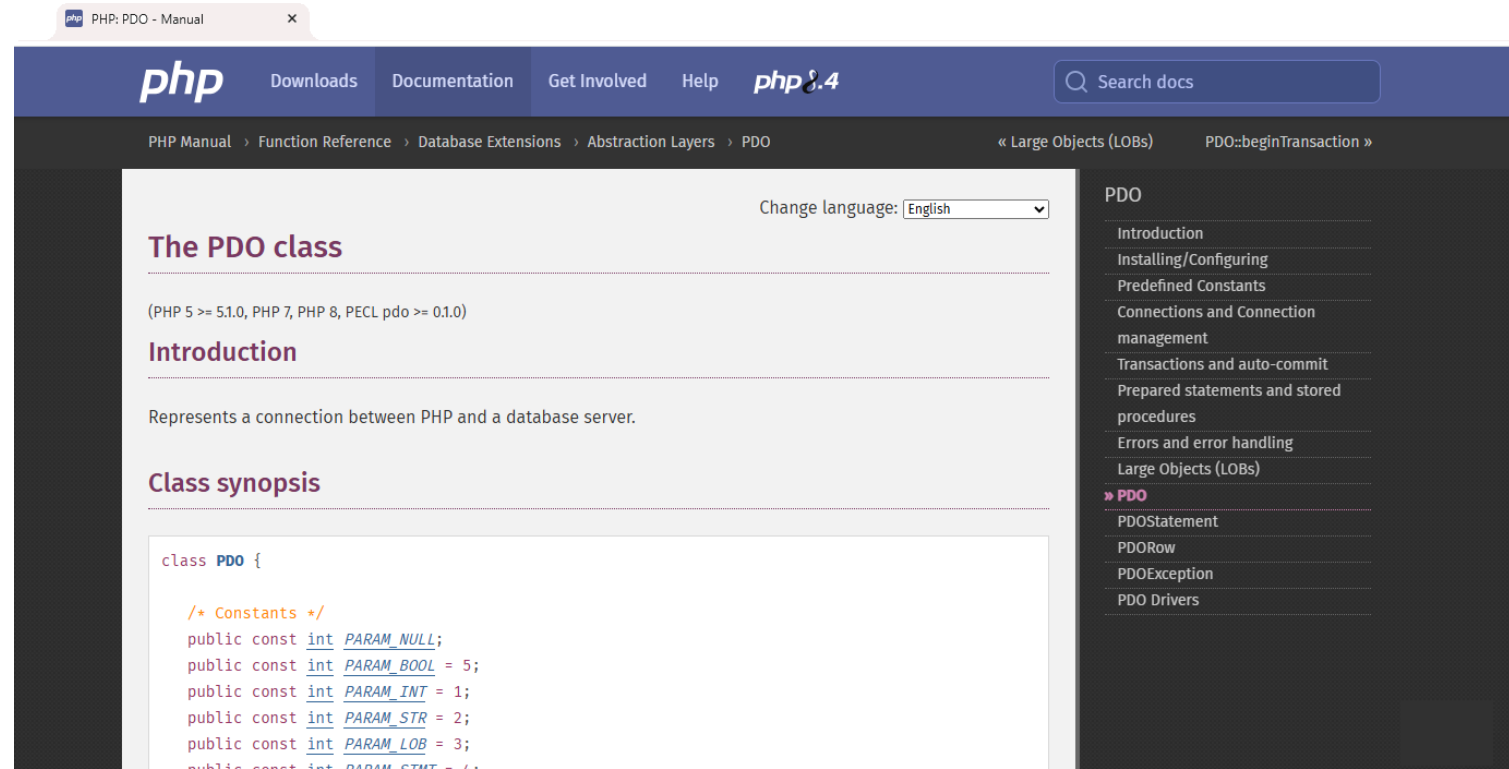
PHP Data Objects

PHP Data Objects

- The PHP Data Objects (PDO) extension defines a lightweight, consistent interface for accessing databases in PHP.
- Each database driver that implements the PDO interface can expose database-specific features as regular extension functions.
 - Note that you cannot perform any database functions using the PDO extension by itself; you must use a database-specific PDO driver to access a database server.
- **PDO provides a data-access abstraction layer.**
 - Regardless of which database you're using, you use the same functions to issue queries and fetch data

PHP Data Objects

- The **PDO** class represents a connection between PHP and a database server.
- More details:



Connections and Connection management

- Connections are established by creating instances of the PDO base class. It doesn't matter which driver you want to use; you always use the PDO class name.
 - The constructor accepts parameters for specifying the database source and optionally for the username and password.

```
<?php

    try {
        $dbconn = new PDO('mysql:host=localhost;dbname=test', 'user', 'password');
    }
    catch (PDOException $e) {
        echo $e->getMessage();
    }

?>
```

Prepared statements

- **Most databases support the concept of prepared statements.**
 - They can be thought of as a kind of compiled template for the SQL that an application wants to run, that can be customized using variable parameters.
- **Prepared statements offer two major benefits:**
 - The query only needs to be parsed (or prepared) once but can be executed multiple times with the same or different parameters.
 - By using a prepared statement, the application avoids repeating the analyze/compile/optimize cycle. This means that prepared statements use fewer resources and thus run faster.
 - The parameters to prepared statements don't need to be quoted (the driver automatically handles this).
 - If an application exclusively uses prepared statements, the developer can be sure that no SQL injection will occur.

Prepared statements

```
<?php
```

```
(...)
```

```
$stmt = $dbcon->prepare("INSERT INTO datatable (name, value) VALUES (?, ?)");  
$stmt->bindParam(1, $name);  
$stmt->bindParam(2, $value);
```

```
// insert one row
```

```
$name = 'one';  
$value = 1;  
$stmt->execute();
```

```
// insert another row with different values
```

```
$name = 'two';  
$value = 2;  
$stmt->execute();
```

```
?>
```